

Recap of Computer Architecture

PARCO — Lezione 2

Architecture model

- Most modern computers are based on the **Von Neumann architecture** (or **Princeton architecture**).
- Also known as «stored-program computer»: both program instructions and data are kept in the same memory.
- In the classical Von Neumann model, instructions and data share the same memory and address space, and are processed by a CPU according to a **sequential instruction stream**.
- The CPU is characterized by the FDE cycle and data path.

Von Neumann Architecture

Four main components: **Memory**; **Control Unit**; **Arithmetic Logic Unit**; **Input/Output**.

RAM Random access memory is used to store both program instructions and data.

- Program instructions: encoded operations interpreted by the processor according to the **instruction set architecture (ISA)**.
- Data: information to be used by the program.
- From the memory perspective they look the same — not from the CPU side.
- Control unit**: coordinates instruction fetch, decode and execution, and controls the movement of operands between registers, memory and execution units.
- Arithmetic Logic Unit**: performs basic arithmetic operations.
- Input/Output**: interface to the human and external environment.

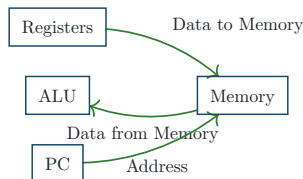


Fig. 1: Von Neumann: CPU e memoria, con data/address bus.

Fetch-Decode-Execute cycle

1. PC identifies the next instruction.
2. Instruction is fetched into the IR.
3. Instruction is decoded.
4. Required operands are obtained.
5. Operation is executed.
6. Architectural state is updated.

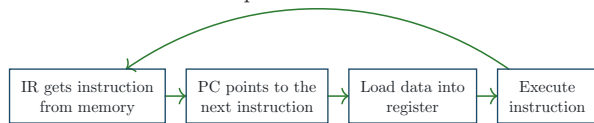


Fig. 2: Ciclo Fetch-Decode-Execute (la control unit è evidenziata nel CPU).

Data Path

- The **ALU** performs arithmetic and bitwise operations on integer binary numbers.
- The **FPU** performs floating-point arithmetic.
- Data movement: register-memory instruction; register-register instruction.

Datapath The datapath is the collection of hardware components that store, move and operate on data, including registers, ALUs, FPUs, load/store units and interconnections.

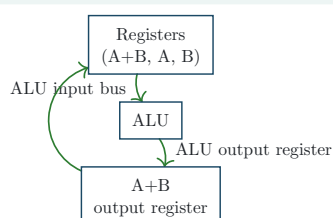


Fig. 3: Datapath ALU: da registri A+B, A, B all'ALU output register.

Different organization

System	Cores
AMD Barcelona	4 cores
Intel Core i7	8 cores
IBM Cell BE	8+1 cores
IBM POWER9	24 cores
Sun Niagara II	8 cores
Nvidia Volta	5120 «cores»
Intel SCC	48 cores, networked
Tilera TILE Gx	100 cores, networked

FUJITSU Processor A64FX

- Superscalar processor of the **out-of-order execution** type.
- Based on Armv8-A architecture and the **Scalable Vector Extension for Armv8-A ISA**.
- Integrates 52 processor cores, including 4 assistant cores.
- Cores organized into 4 **Core Memory Group (CMG)**.

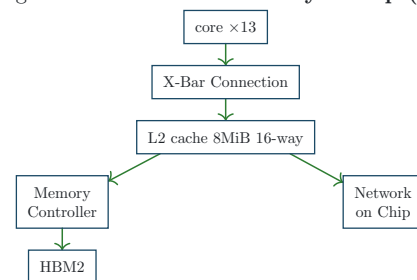


Fig. 4: A64FX: configurazione di un CMG (13 core, X-Bar, L2, HBM2, NoC).
Chip configuration: 4 CMG; Tofu Controller; PCIe Controller; HBM2 (×4); Network on Chip (**Ring bus**).

GPU and accelerators

NVIDIA AMPERE GA102 GPU ARCHITECTURE.

Componente	Ruolo
PCI Express 4.0 Host Interface	interfaccia host
GigaThread Engine	scheduling dei thread
GPC	Graphics Processing Cluster
L2 Cache	cache condivisa
Memory Controllers	interfacciamento memoria
High-Speed Hub	interconnessione
NVLink	Four ×4 Links

Detail of a **Stream Multiprocessor (SM)**: unità FP32, unità FP32/INT32, Tensor cores, RT cores, L1 cache/shared memory; **2nd Generation RT Core**.

Emerging Systems

- Processing Using Memory (PUM)**: minimal chip modification to support Bulk Bitwise Operations [Seshadri17].
- Processing near memory (PNM)**: example adding CU in the stack layer.

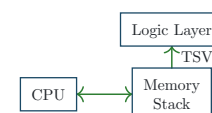


Fig. 5: 3D stacked DRAM (logic layer, TSV) e PIM sul memory controller / DIMM. Componenti: logic layer, memory controller, memory module (DIMM); PIM nel memory controller o sulla DIMM.

Flynn's Taxonomy of Computers

Mike Flynn, «Very High-Speed Computing Systems,» Proc. of IEEE, 1966.

SISD	Single instruction operates on single data element
SIMD	Single instruction operates on multiple data elements; array processor, vector processor
MISD	Multiple instructions operate on single data element — rare in general-purpose computing; practical examples are uncommon and classification is sometimes application-dependent
MIMD	Multiple instructions operate on multiple data elements (multiple instruction streams); multiprocessor, multithreaded processor

Other classifications exist beyond this taxonomy.

Single Instruction on a single Data

- Simple serial architecture.
- **Single Instruction**: a single logical instruction stream controls execution.
- **Single Data**: only one data stream is being used as input during any one clock cycle.
- In-order execution (modern architectures support out-of-order).
- Examples: older generation mainframes, minicomputers, workstations and single processor/core PCs.

Latency and Throughput

Latency Number of clock cycles an instruction takes to have its data available for use by another instruction (lower is better). It also represents the cost to get the next instruction.

Throughput Number of instructions, operations, or tasks completed per unit time (higher is better).

- A design may aim to minimise latency, maximise throughput, or balance both.
- The concepts of latency and throughput are also applied to CPU and memory.

Understanding the limits of Von Neumann architectures

Performance in VN architectures depends on many factors (all together):

- FDE design and ISA (latency)
- ALU technology and PEs organization (math bandwidth)
- Memory sub-system (latency and memory bandwidth)
- Applications (ratio between math operations and read/write operations)

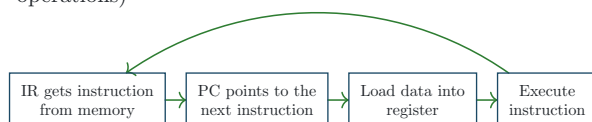


Fig. 6: Analisi del potenziale bottleneck nel ciclo FDE.

Fetch Decode Execution Design

- Processors run at a constant rate (**clock frequency**).
- Performance also depends on the ability to execute instructions per second (MIPS) via the FDE cycle.
- Different instructions may require different numbers of cycles; the average cost is reflected in **CPI**.
- For numerical workloads, **FLOP/s** is often used: it measures application-relevant floating-point work rather than instruction count.

Performance Depends on the task and is influenced by: **clock cycle time** (HW technology, system organisation); **CPI** (system organisation, ISA); **instruction count IC** (ISA, compiler technology, benchmark program).

CPU time $\text{CPU time} = \text{IC} \times \text{Cycles per instruction} \times \text{Clock cycle time}$

Implicit Parallelism

Implicit parallelism is parallel execution extracted automatically by the compiler and/or hardware without the programmer explicitly creating parallel tasks or threads. It contrasts with **explicit parallelism**, where the programmer specifically indicates parallel sections in the code.

Explicit parallelism, where the programmer specifically indicates parallel sections in the code.

- Historically, increasing clock frequency was a major source of performance improvement, but power and thermal constraints limited continued frequency scaling.
- Higher levels of device integration have made many transistors available; how best to utilise these resources is an important question.

Parallelism on the hardware level:

- Current processors use these resources in multiple functional units and execute multiple operations in the same cycle (Instruction Level Parallelism and Superscalar Architectures).
- The way in which instructions are selected and executed provides different architectures (out-of-order execution and pipelining).
- Vectorization.

Instruction Level Parallelism (ILP)

ILP Ability to execute multiple independent instructions from a single instruction stream concurrently.

- Introduction of an **execution unit** (also called a **functional unit**): performs the operations (load and store and arithmetic operations) as instructed by the program. It may have its own internal control sequence unit — not to be confused with the CPU's main control unit — and some registers.
- Components: arithmetic logic unit (ALU), address generation unit (AGU), floating-point unit (FPU), load-store unit (LSU), branch execution unit (BEU).
- Superscalar architectures have many EUs.

Example of Superscalar execution

A **superscalar processor** is a CPU that implements a form of parallelism called instruction-level parallelism within a single processor. It can find instruction dependencies.

$$a = x * x + y * y + z * z$$

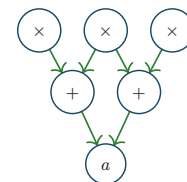


Fig. 7: Albero delle dipendenze di $a = x * x + y * y + z * z$.

Livelli: ILP = 3 sulla riga dei tre prodotti; ILP = 1 sul primo add; ILP = 1 sull'add finale.

```
mul r1, r0, r0    <- dependent instructions
mul r1, r1, r1
st  r1, mem[r2]
...
add r0, r0, r3    <- independent instructions
add r1, r4, r5
...
```

Modern architectures

- In Flynn's taxonomy, a single-core superscalar processor is classified as an **SISD**.
- Superscalar design can be combined with other performance enhancement techniques:
 - **Speculative execution**: predict results to continue execution.
 - **Out-of-Order (OoO) execution**: potentially change execution order of instructions.
 - **Pipeline**.
 - **Vectorization** (SIMD approach).

Pipeline

Goal: **reduce the latency and maximize the use of the resource**. Washing clothes example (credits ETH Introduction to Parallel Programming). Inputs are instruction streams.

Balanced Pipeline All the steps require the same time (practically never happens).

Bounding Throughput bound $= 1 / \max(\text{computation time (stages)})$
Latency bound $= \text{num stages} \times \max(\text{computation time (stages)})$

Time	T0	T1	T2	T3
1st batch (Input 0)	w	d	f	c
2nd batch (Input 1)	—	w	d	f

Fig. 8: Pipeline a 4 stadi: due batch (Input 0, Input 1) in ingresso.

Inputs	T0	T1	T2	T3	T4	T5	T6	T7	T8	T9
I0	S0	S1	S2	S3						
I1		S0	S1	S2	S3					
I2			S0	S1	S2	S3				
I3				S0	S1	S2	S3			
I4					S0	S1	S2	S3		
I5						S0	S1	S2	S3	
I6							S0	S1	S2	S3

Fig. 9: Space-time diagram di un pipeline bilanciato a 4 stadi.

Fasi del diagramma: **Lead In** → **Full Utilization** → **Lead out**.

Pipeline non bilanciato

Stadi: w Washer 5 s; d Dryer 10 s; f Folding 5 s; c Closet 10 s. Assumendo 5 washing loads, il tempo totale è **70 s**. Latency issue; soluzione: fare in modo che ogni stadio richieda quanto il più lungo.

Time (s)	0	5	10	15	20	25	30	35	40	45	50	55	60	65	70
Load 1		w	d	d	f	c	c								
Load 2			w	—	d	d	f	c	c						
Load 3				w	—	—	d	d	f	c	c				
Load 4					w	—	—	—	d	d	f	c	c		
Load 5						w	—	—	—	—	d	d	f	c	c

Fig. 10: Schedule non bilanciato (5 carichi).

Bilanciamento degli stadi

Portando ogni stadio a 10 s, il tempo totale è **80 s**. Latency bound: 40 s per load. Throughput: 1 load / 10 s ≈ 6 loads / minuto. Domanda: si possono migliorare latency e throughput insieme?

Aumentare le FU

- w 5 s; due dryer in serie: d1 4 s, d2 6 s; f 5 s; due closet in serie: c1 4 s, c2 6 s.
 - Bounding della latency: 6 s. Ogni d1/d2 e ogni c1/c2 richiede 6 s.
- Tempo totale **60 s**. Latency bound: 36 (6×6) s per load. Throughput: 1 load / 6 s ≈ 10 loads / minuto.

Time (s)	0	6	12	18	24	30	36	42	48	54	60	110
Load 1		w	d1	d2	f	c1	c2					
Load 2			w	d1	d2	f	c1	c2				
Load 3				w	d1	d2	f	c1	c2			
Load 4					w	d1	d2	f	c1	c2		
Load 5						w	d1	d2	f	c1	c2	

Fig. 11: Schedule bilanciato: dryer e closet sdoppiati (d1/d2, c1/c2).

DA VERIFICARE L'ultima intestazione della tabella è 110 s, incoerente con il passo di 6 s (atteso 66 s); trascritta come nel render.

Pipeline a 5 stadi

- Fetching of instructions from memory is a major bottleneck in instruction execution speed.
- Instructions can be stored in a set of registers called the **pre-fetch buffer**.
- Execution is divided into two parts: fetching and actual execution.
- Often divided into many parts, each handled by a dedicated piece of hardware, all of which can run in parallel.



Fig. 12: Pipeline a 5 stadi (a) struttura, (b) diagramma temporale.

S1:	1	2	3	4	5	6	7	8	9
S2:		1	2	3	4	5	6	7	8
S3:			1	2	3	4	5	6	7 ...
S4:				1	2	3	4	5	6
S5:					1	2	3	4	5
	1	2	3	4	5	6	7	8	9